

Performance Testing

Date	30 June 2026
Team ID	XXXXXX
Project Name	Rising Waters – AI-Powered Flood Prediction and Early Warning System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	unittest, Postman
Type of Testing	Unit, Functional, Model, Performance & Deployment Testing
Target Module / API	Flask Routes, Prediction API, XGBoost Classifier
Test Environment	Local Flask Server & Vercel Cloud
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Unit Test: Flask routes (Home, Predictor, Portal)	1	5	HTTP Status 200 OK for all active routes
2	Unit Test: Prediction API endpoint (/predict)	1	5	Valid JSON response with risk level and probability
3	Functional Test: Meteorological parameter validation	1	10	Returns HTTP 400 Bad Request for out-of-bounds inputs
4	Model Testing: XGBoost classifier validation	1	10	Accuracy on test set is 96.55% with low latency
5	Performance Test: Response time under concurrent requests	25	60	Avg response time < 500ms with 0% error rate
6	Deployment Test: Vercel serverless latency & logging	5	30	Stable deployment, prediction runs saved in logs

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.18 sec	Pass	Very fast prediction latency
2	Response Time (Max)	< 5 seconds	0.65 sec	Pass	Within limit under peak load
3	Prediction Accuracy	> 90%	96.55%	Pass	XGBoost model achieved high precision
4	Throughput (Req/sec)	> 10 Req/sec	35 Req/sec	Pass	Stable serverless capacity
5	Error Rate	< 1%	0%	Pass	All inputs validated correctly
6	CPU Utilization	< 80%	15%	Pass	Low resource consumption

Step 4: Observations & Analysis

Step 5: Screenshots / Evidence